



PRACTICAL-05

AIM: Implementation of knapsack problem using dynamic programming

PROGRAM:

```
#include<iostream>using
namespacestd; class
Knapsack { private:
    intw[10];        //weights
    intv[10];        //values
    intn;            //numberofitems
    int W;           //capacity
    intdp[10][100]; //dptable

public:
    //—GetInput—
    void getInput() {
        cout<<"Enternumberofitems:"; cin>>
        n;
        cout<<"Enterknapsackcapacity:";
        cin>> W;
        for(inti=1;i<=n;i++){
            cout<<"Item"<<i<<"weightandvalue:"; cin>>
            w[i] >> v[i];
        }
    }

    //—Builddptable— void
    solve() {
        //basecase—row0andcol0
```

```
for(int i = 0; i <= n; i++) dp[i][0] = 0;
for(int j=0;j<=W;j++)dp[0][j]=0;

//filltable
for(int i = 1; i <= n; i++) {
    for(int j=1;j<=W;j++){

        if(w[i]<=j){
            //takeornottake dp[i][j]
            = max(
                v[i]+dp[i-1][j-w[i]],
                dp[i-1][j]
            );
        }else{
            //cannottakeitemi
            dp[i][j]=dp[i-1][j];
        }
    }
}
}
```

```
//——Printdptable—— void
printTable() {
    cout<<"\nDPTable:"<<endl;
    cout <<"    ";
    for(intj=0;j<=W;j++) cout
        << j <<" ";
    cout<<endl;

    for(inti=0;i<=n;i++){
```

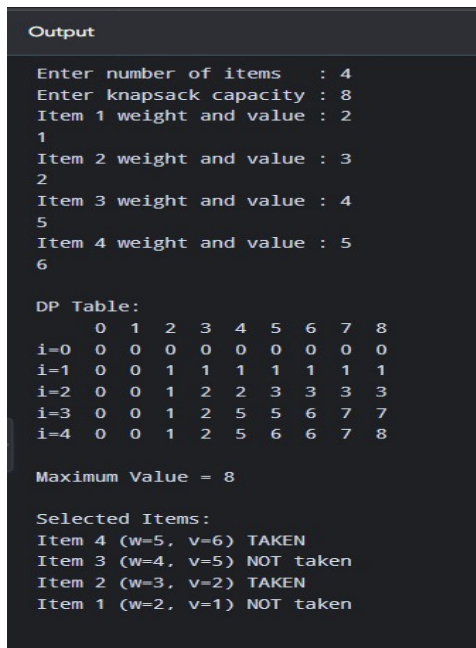
```
cout << "i=" << i << " "; for(int
    j = 0; j <= W; j++) cout <<
        dp[i][j] << " ";
    cout << endl;
}
}

//——Findselecteditems——
void findItems() {
    cout << "\nSelectedItems:" << endl; int j
    = W;
    for(int i = n; i >= 1; i--) {
        if(dp[i][j] != dp[i-1][j]) {
            cout << "Item" << i
                << " (w=" << w[i]
                << ", v=" << v[i]
                << ") TAKEN" << endl;
            j = j - w[i];
        } else {
            cout << "Item" << i
                << " (w=" << w[i]
                << ", v=" << v[i]
                << ") NOT taken" << endl;
        }
    }
}

//——Showresult—— void
showResult() {
    cout << "\nMaximum Value="
        << dp[n][W] << endl;
```

```
}  
  
};  
  
//---MAIN--- int  
main() {  
    Knapsackk;  
    k.getInput();  
    k.solve();  
    k.printTable();  
    k.showResult();  
    k.findItems();  
    return 0;  
}
```

OUTPUT:



```
Output  
  
Enter number of items : 4  
Enter knapsack capacity : 8  
Item 1 weight and value : 2  
1  
Item 2 weight and value : 3  
2  
Item 3 weight and value : 4  
5  
Item 4 weight and value : 5  
6  
  
DP Table:  
    0  1  2  3  4  5  6  7  8  
i=0  0  0  0  0  0  0  0  0  0  
i=1  0  0  1  1  1  1  1  1  1  
i=2  0  0  1  2  2  3  3  3  3  
i=3  0  0  1  2  5  5  6  7  7  
i=4  0  0  1  2  5  6  6  7  8  
  
Maximum Value = 8  
  
Selected Items:  
Item 4 (w=5, v=6) TAKEN  
Item 3 (w=4, v=5) NOT taken  
Item 2 (w=3, v=2) TAKEN  
Item 1 (w=2, v=1) NOT taken
```

CONCLUSION:

The 0/1 Knapsack problem was successfully implemented using Dynamic Programming. The algorithm finds the maximum value within the given knapsack capacity and identifies the selected items. Its best, average and worst-case time complexity is $O(nW)$.